



CS 305 Project One Template

Document Revision History

Version	Date	Author	Comments
1.0	5/24/2026	Tyrone Grant	

Client



Instructions

Submit this completed vulnerability assessment report. Replace the bracketed text with the relevant information. In this report, identify your security vulnerability findings and recommend the next steps to remedy the issues you have found.

- Respond to the five steps outlined below and include your findings.
- Respond using your own words. You may also include images or supporting materials. If you include them, make certain to insert them in the relevant locations in the document.
- Refer to the Project One Guidelines and Rubric for more detailed instructions about each section of the template.



Developer

Tyrone Grant

1. Interpreting Client Needs

Determine your client's needs and potential threats and attacks associated with the company's application and software security requirements. Consider the following questions regarding how companies protect against external threats based on the scenario information:

- What is the value of secure communications to the company?
- Are there any international transactions that the company produces?
- Are there governmental restrictions on secure communications to consider?
- What external threats might be present now and in the immediate future?
- What modernization requirements must be considered, such as the role of open-source libraries and evolving web application technologies?

Artemis Financial is a consulting company that develops individualized financial plans for its customers, plans that include savings, retirement, investment, and insurance. Due to the sensitive nature of the financial information that the institution handles, secure communication is of the utmost priority to the company as well as its customers. Artemis Financial doesn't directly state that they conduct business internationally, nor was there any evidence in the code that would suggest that they have an international customer base.

The government regulates certain aspects of secure communications as they pertain to financial institutions, things like data encryption and localization, and consumer privacy mandates are imposed by entities like the FTC or the FDIC. These regulations are in place because of the threat level that financial institutions are subject to. Threats such as phishing attempts, identity theft, scams, and data breaches are pathways that bad actors may use to harm Artemis Financial or its customers. Today, many applications use open-source libraries to integrate complex features. Special care must be taken by financial institutions due to industry standards and involved risk to protect stakeholders and users. This means that enterprise level applications must carefully vet any open-source dependencies before integration. As web application technologies continue to evolve, the environment requires increased knowledge and vigilance over the widened attack surface that these technologies present.

2. Areas of Security

Refer to the vulnerability assessment process flow diagram. Identify which areas of security apply to Artemis Financial's software application. Justify your reasoning for why each area is relevant to the software application.

Input Validation: The application will manage inputs such as login credentials, deposit information, personal sensitive data, and transaction details. These inputs must be validated to prevent errors or exploitation.

APIs: The application utilizes REST API to conduct client/server operations. This communication must be verifiably secure.

Cryptography: The government requires that certain consumer information is encrypted, and cryptography helps to protect information transmitted over the internet and login credentials.



Client/Server: Client/server communications will be handled via REST API.

Code Error: The application must implement proper exception handling to catch and log errors without exposing sensitive system information

Code Quality: The application must follow modern coding principles and abide by industry standards. Poor code quality practices directly increase the attack surface of the program.

Encapsulation: Data structures must be secure to prevent unmitigated manipulation, improper or unauthorized access.

3. Manual Review

Continue working through the vulnerability assessment process flow diagram. Identify all vulnerabilities in the code base by manually inspecting the code.

Customer.java - The account balance variable isn't private, so it can be accessed externally and potentially altered.

Customer.java - The deposit method doesn't contain a check for amounts that are out of bounds, such as negative amounts or restricted limits.

myDateTime.java - The variables that are declared initially are not private, so they may be accessed externally.

myDateTime.java - The set time method in the class doesn't contain a check for out of bound times.

DocData.java - The class's key and value parameters in the read document method aren't checked to make sure that their values are appropriate, which may leave the program vulnerable to SQL injections.

DocData.java - The database connection in the read document method is never closed after actions are done. This leads to potential data leakage and connection issues.

CRUD.java - There appears to be duplicate code in the class. The second "crud" method handles the information appropriately, so the first can be deleted.

4. Static Testing

Run a dependency check on Artemis Financial's software application to identify all security vulnerabilities in the code. Record the output from the dependency-check report. Include the following items:

- The names or vulnerability codes of the known vulnerabilities
- A brief description and recommended solutions provided by the dependency-check report
- Any attribution that documents how this vulnerability has been identified or documented previously

Project: rest-service

com.tekrest-service:0.0.1-SNAPSHOT

Scan Information [\(show all\)](#)

- dependency-check version: 12.2.2
- Report Generated On: Sun, 24 May 2026 22:49:50 -0400
- Dependencies Scanned: 38 (20 unique)
- Vulnerable Dependencies: 11
- Vulnerabilities Found: 190
- Vulnerabilities Suppressed: 0
- ...

Summary

Summary of Vulnerable Dependencies [\(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncycastle-crypto-package:1.46:***** cpe:2.3:a:bouncycastle:bouncycastle_crypto_package:1.46:***** cpe:2.3:a:bouncycastle:bouncycastle_for_jdk15on:1.46:***** cpe:2.3:a:bouncycastle:legion-of-the-bouncycastle-java-cryptography-api:1.46:***** cpe:2.3:a:bouncycastle:the_bouncycastle_crypto_package_for_java:1.46:*****	pkg:maven/org.bouncycastle:bcprov-jdk15on@1.46	HIGH	4	Highest	38
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:***** cpe:2.3:a:hibernate:hibernate_validator:6.0.18:*****	pkg:maven/org.hibernate.validator:hibernate-validator@6.0.18.Final	MEDIUM	3	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:***** cpe:2.3:a:fasterxml:jackson-databind:2.10.2:***** cpe:2.3:a:fasterxml:jackson-modules-jv8:2.10.2:*****	pkg:maven/com.fasterxml.jackson.core:jackson-databind@2.10.2	HIGH	6	Highest	39
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*****	pkg:maven/org.apache.logging.log4j:log4j-api@2.12.1	MEDIUM	3	Highest	42
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*****	pkg:maven/ch.qos.logback:logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml:snakeyaml:1.25:*****	pkg:maven/org.yaml:snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*****	pkg:maven/org.springframework.boot:spring-boot@2.2.4.RELEASE	CRITICAL	8	Highest	39
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:spring:spring_framework:5.2.3:release:***** cpe:2.3:a:spring:spring_framework:5.2.3:release:***** cpe:2.3:a:vmware:spring_framework:5.2.3:release:*****	pkg:maven/org.springframework:spring-core@5.2.3.RELEASE	CRITICAL*	17	Highest	36
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:spring:spring_framework:5.2.3:release:***** cpe:2.3:a:spring:spring_framework:5.2.3:release:***** cpe:2.3:a:vmware:spring_framework:5.2.3:release:*****	pkg:maven/org.springframework:spring-web@5.2.3.RELEASE	CRITICAL*	18	Highest	34
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:***** cpe:2.3:a:apache:tomcat:9.0.30:*****	pkg:maven/org.apache.tomcat.embed:tomcat-embed-core@9.0.30	CRITICAL*	60	Highest	30
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:***** cpe:2.3:a:apache:tomcat:9.0.30:*****	pkg:maven/org.apache.tomcat.embed:tomcat-embed-websocket@9.0.30	CRITICAL*	61	Highest	30

Dependencies & Associated Codes

bcprov-jdk15on-1.46.jar: CVE-2016-1000338, CVE-2020-15522, CVE-2023-33202, CVE-2015-7940

hibernate-validator-6.0.18.Final.jar: CVE-2025-35036, CVE-2020-5245, CVE-2025-4428, CVE-2023-1932, CVE-2020-10693

jackson-databind-2.10.2.jar: CVE-2020-25649, CVE-2020-36518, CVE-2021-46877, CVE-2022-42003, CVE-2022-42004, CVE-2023-35116

log4j-api-2.12.1.jar: CVE-2026-34479, CVE-2026-34477, CVE-2025-68161, CVE-2020-9488

logback-core-1.2.3.jar: CVE-2023-6378, CVE-2021-42550

snakeyaml-1.25.jar: CVE-2022-1471, CVE-2017-18640, CVE-2003-1564, CVE-2022-25857, CVE-2022-38749, CVE-2022-38750, CVE-2022-38751, CVE-2022-38752, CVE-2022-41854

spring-boot-2.2.4.RELEASE.jar: CVE-2023-20873, CVE-2022-27772, CVE-2023-20883, CVE-2026-22733, CVE-2026-40972, CVE-2026-40973, CVE-2026-40975, CVE-2026-40977

spring-core-5.2.3.RELEASE.jar: CVE-2015-5211, CVE-2020-5421, CVE-2021-22060, CVE-2021-22096, CVE-2021-22118, CVE-2022-22950, CVE-2022-22965, CVE-2022-22968, CVE-2022-22970, CVE-2022-22971, CVE-2023-20861, CVE-2023-20863, CVE-2024-22243, CVE-2024-22259, CVE-2026-22735, CVE-2026-22737, CVE-2026-22740, CVE-2026-22741, CVE-2026-22745

spring-web-5.2.3.RELEASE.jar: CVE-2016-1000027, CVE-2015-5211, CVE-2020-5421, CVE-2021-22060, CVE-2021-22096, CVE-2021-22118, CVE-2022-22950, CVE-2022-22965, CVE-2022-22968, CVE-2022-22970, CVE-2022-22971, CVE-2023-20861, CVE-2023-20863, CVE-2024-22243, CVE-2024-22259, CVE-2026-22735, CVE-2026-22737, CVE-2026-22740, CVE-2026-22741, CVE-2026-22745

tomcat-embed-core-9.0.30.jar: CVE-2019-17569, CVE-2020-1935, CVE-2020-1938, CVE-2020-9484, CVE-2020-9494, CVE-2020-11996, CVE-2020-13934, CVE-2020-13935, CVE-2020-13943, CVE-2020-17527,



CVE-2021-24122, CVE-2021-25122, CVE-2021-25329, CVE-2021-30640, CVE-2021-33037, CVE-2021-41079, CVE-2021-43980, CVE-2022-29885, CVE-2022-34305, CVE-2022-42252, CVE-2023-28708, CVE-2023-41080, CVE-2023-42795, CVE-2023-44487, CVE-2023-45648, CVE-2023-46589, CVE-2024-21733, CVE-2024-23672, CVE-2024-24549, CVE-2024-34750, CVE-2024-38286, CVE-2024-50379, CVE-2024-52316, CVE-2024-54677, CVE-2024-56337, CVE-2025-24813, CVE-2025-31651, CVE-2025-46701, CVE-2025-48988, CVE-2025-48989, CVE-2025-49124, CVE-2025-49125, CVE-2025-52434, CVE-2025-52520, CVE-2025-53506, CVE-2025-55668, CVE-2025-55752, CVE-2025-61795, CVE-2025-66614, CVE-2026-24733, CVE-2026-24880, CVE-2026-25854, CVE-2026-29146, CVE-2026-34487, CVE-2026-41284, CVE-2026-41293, CVE-2026-42498, CVE-2026-43512, CVE-2026-43513, CVE-2026-43514, CVE-2026-43515

tomcat-embed-websocket-9.0.30.jar: Includes all codes associated with the prior dependency with the addition of CVE-2020-8022.

5. Mitigation Plan

Interpret the results from the manual review and static testing report. Then identify the steps to mitigate the identified security vulnerabilities for Artemis Financial's software application.

Dependencies & Associated Codes:

The most critical vulnerabilities appear to stem from **snakeyaml-1.25.jar**, **spring-boot-2.2.4.RELEASE.jar**, **spring-core-5.2.3.RELEASE.jar**, **spring-web-5.2.3.RELEASE.jar**, **tomcat-embed-core-9.0.30.jar**, and **tomcat-embed-websocket-9.0.30.jar**. These libraries appear to be fairly out of date, so using the most recent version of each library may help reduce the encountered dependencies where vulnerabilities have been addressed in later updates. I would suggest using the most compatible version of Java as well as updated versions of **bcprov-jdk15on-1.46.jar**, **jackson-databind-2.10.2.jar** and **logback-core-1.2.3.jar** to mitigate some of the high severity dependencies.

Manual Review:

To address the vulnerabilities observed after a manual inspection, the code base requires refactoring to ensure that customer data is managed and administered securely. Some alterations include adding checks to method parameters, ensuring certain variables are private to prevent external access, and closing database connections after entrance.